# FY-26: Career Preview Program project Review (CPP-3)

Updated June 2026

College /University Name: Jaipur Engineering College & Research Centre

# AGENTIC BUG TRIAGE & ROUTING

**Project Name :** Agentic Bug Triage & Routing

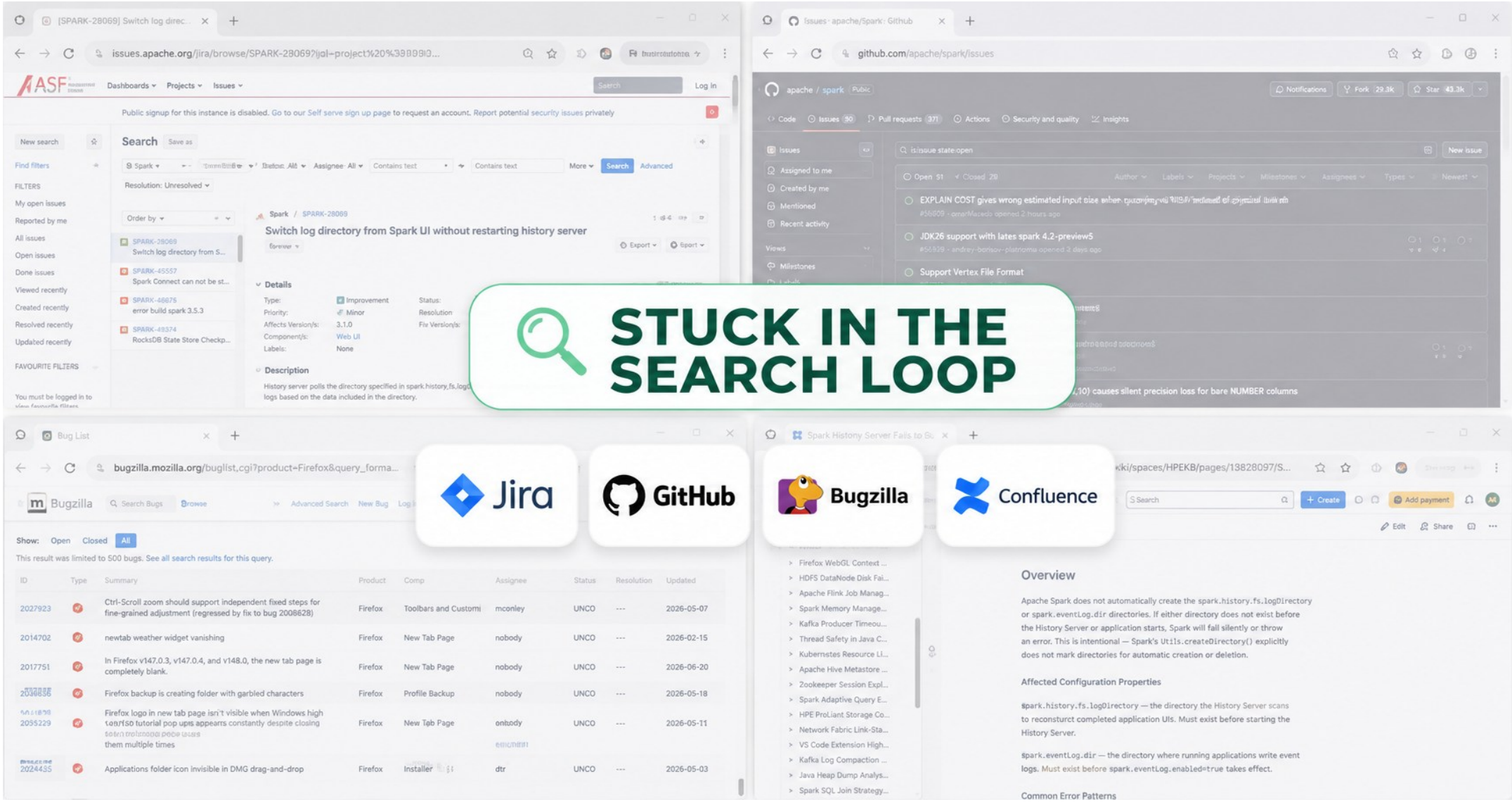
**College / University Name :** Jaipur Engineering College & Research Centre

**Project Participants :** Anuj Modani
Pulkit Jain
Shivansh Gaur
Disha Jain
Om Jain

**Mentor Names :** Madhukar Rupakumar, Divakar Padiyar, PE Rajesh, Bharath Narayan (HPE)
Aneesh Kumar Mishra (College Mentor)

**GitHub Link :**
Source Code: https://github.com/pulkitjn3010/agentic-bug-triage-and-routing
Project Documents: https://github.com/sh1vanshgaur/Agentic-Bug-Triage

# Problem Demonstration




**Problem Demonstration** : https://youtu.be/gOyspIhLMJc

# Problem Statement

## 1. Multi-System Data Fragmentation

Bug data is spread across multiple systems with no unified view

## 2. Manual Context Gathering

Engineers manually search tickets, logs, and customer cases.

## 3. Missing Cross-System Correlation

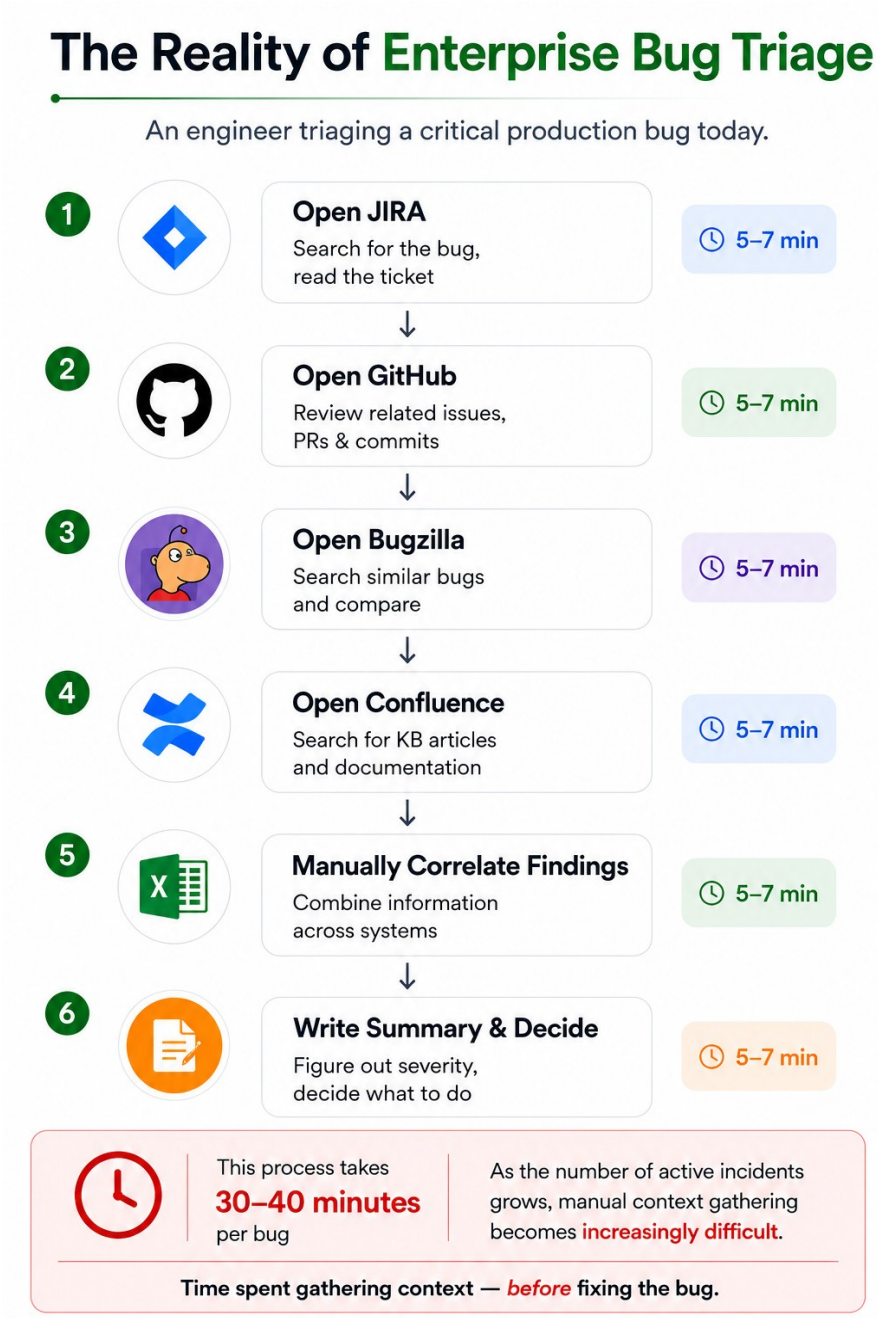
Related bugs across systems are not linked automatically.

## 4. Scattered Knowledge Repositories

Runbooks and workarounds remain buried in knowledge bases.

## 5. Lack of Actionable Triage Intelligence

Trackers provide no AI-driven severity, root cause, or recommendations.



# Solution Overview

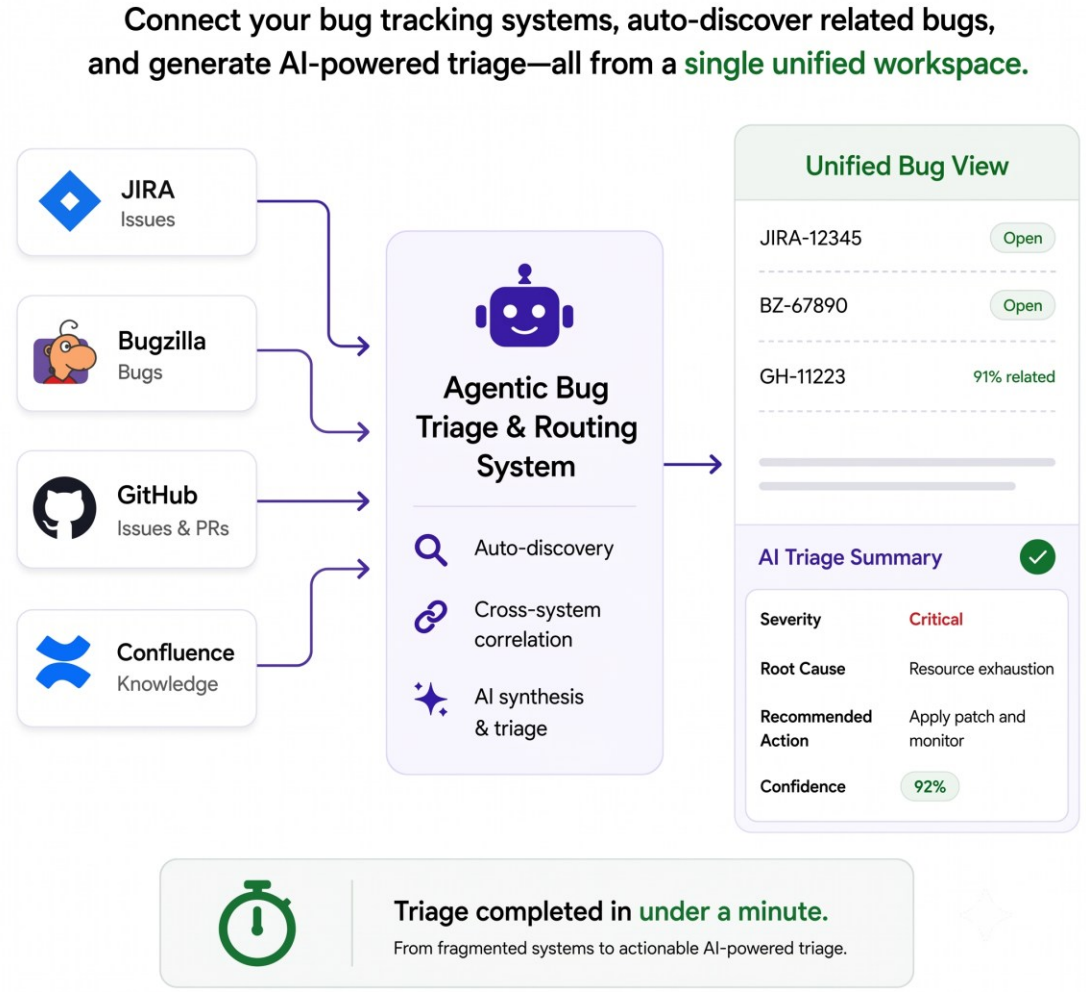
## 1. Unified Bug View
Bring Jira, GitHub, Bugzilla and Confluence into one workspace.

## 2. Automated Context Gathering
Collect bug details, metadata and logs automatically.

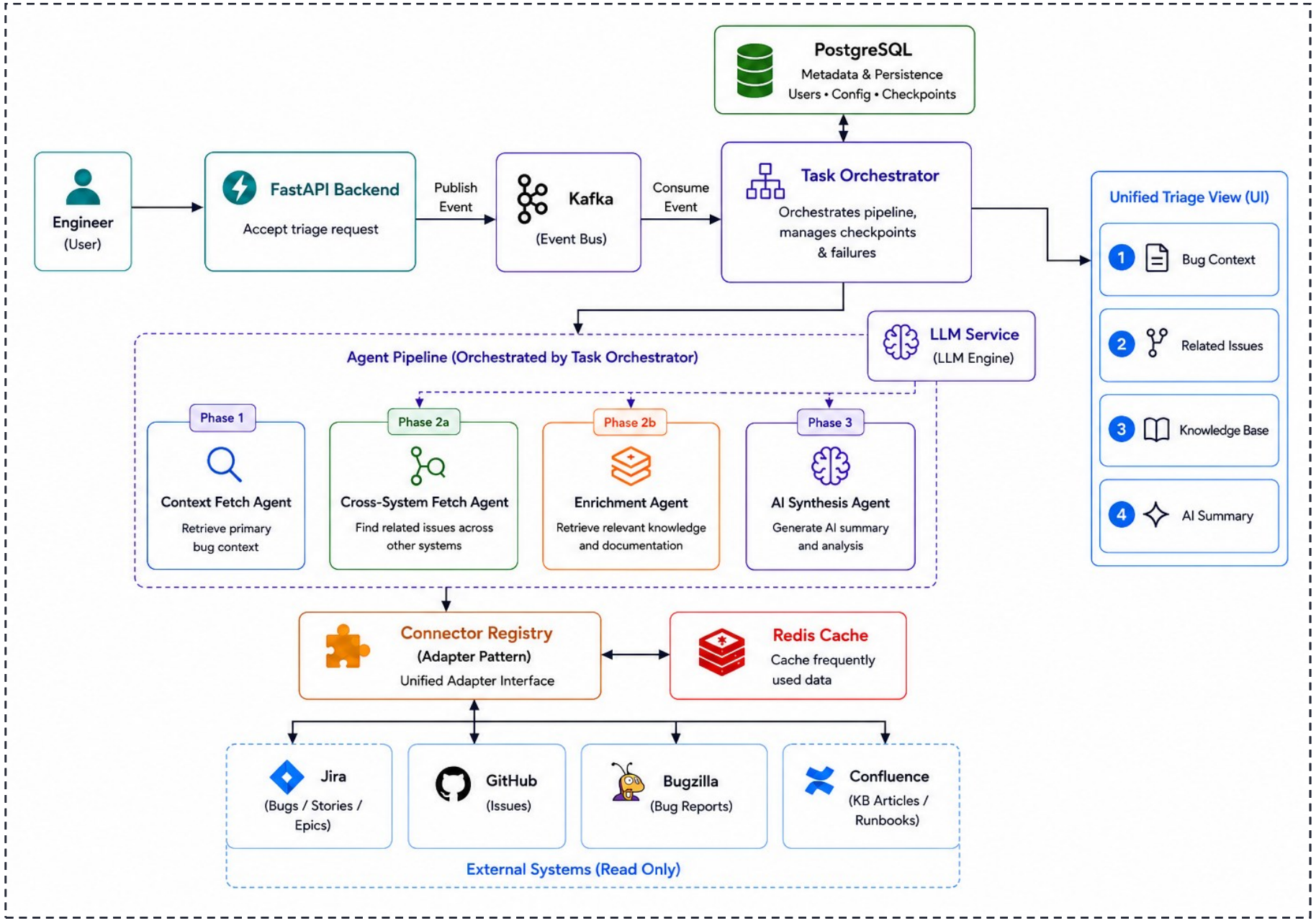
## 3. Cross-System Correlation
Related bugs across systems are linked automatically.

## 4. Knowledge Base Enrichment
Retrieve relevant runbooks, documentation and historical fixes.

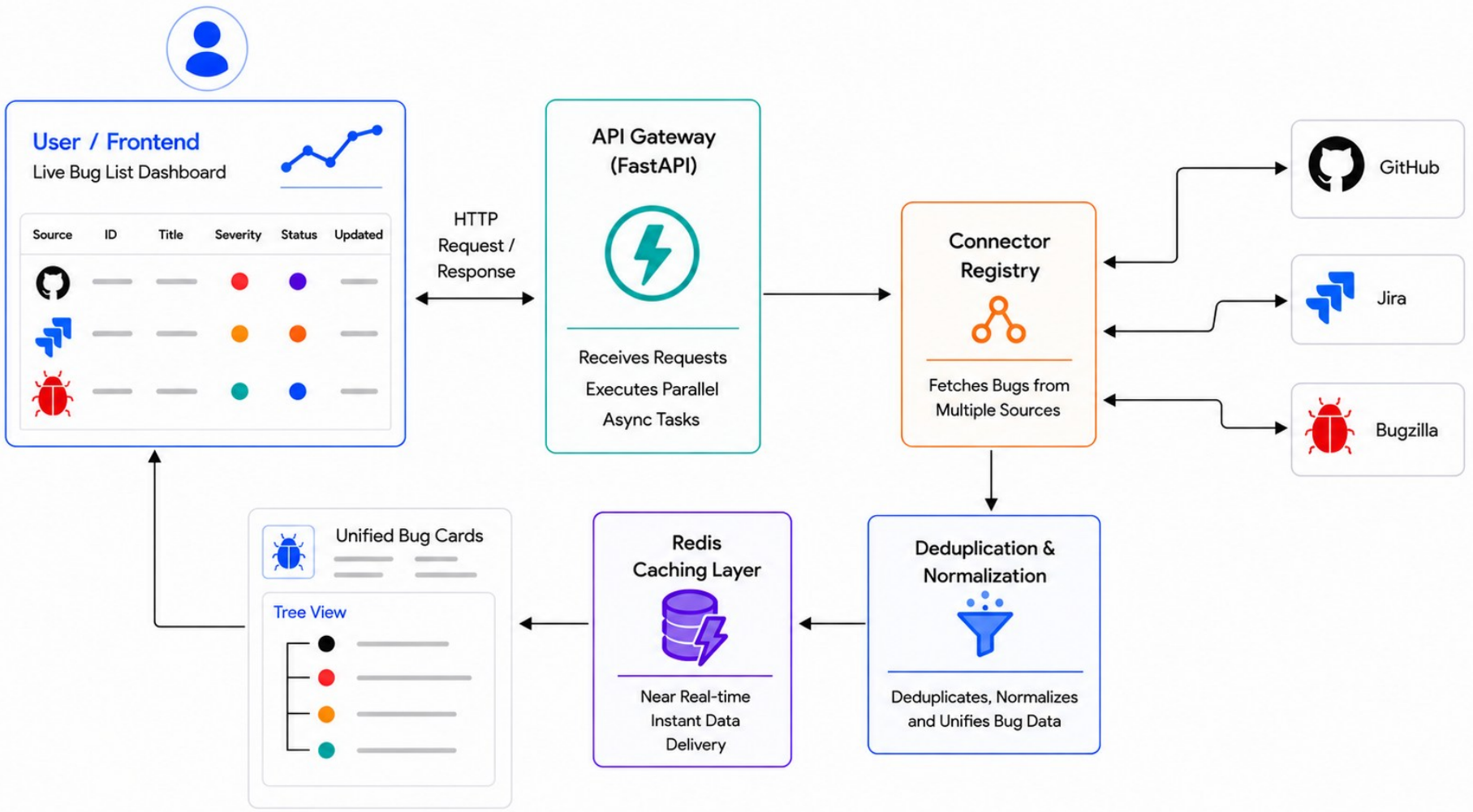
## 5. AI Powered Triage
Generate severity, root cause, confidence score and recommended actions.



# System Architecture



# Live Bug Discovery Pipeline




## Parallel Fetching

- Async API calls
- Multi-source aggregation

## In memory filtering

- Instantaneous search and filtering

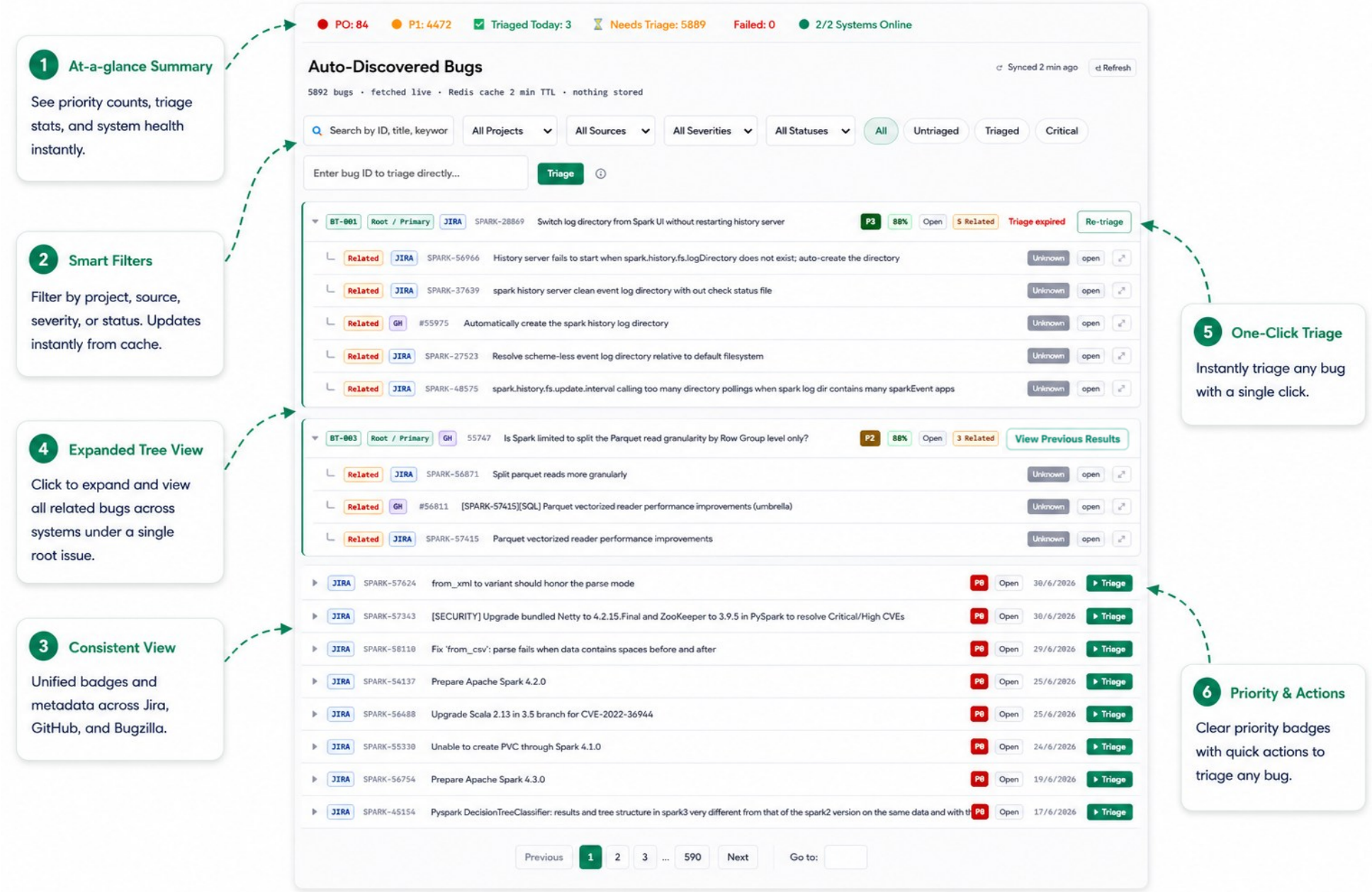
## Dashboard

- Live bug updates Unified bug cards

## Tree View

- Related bug discovery across multiple tracking systems.

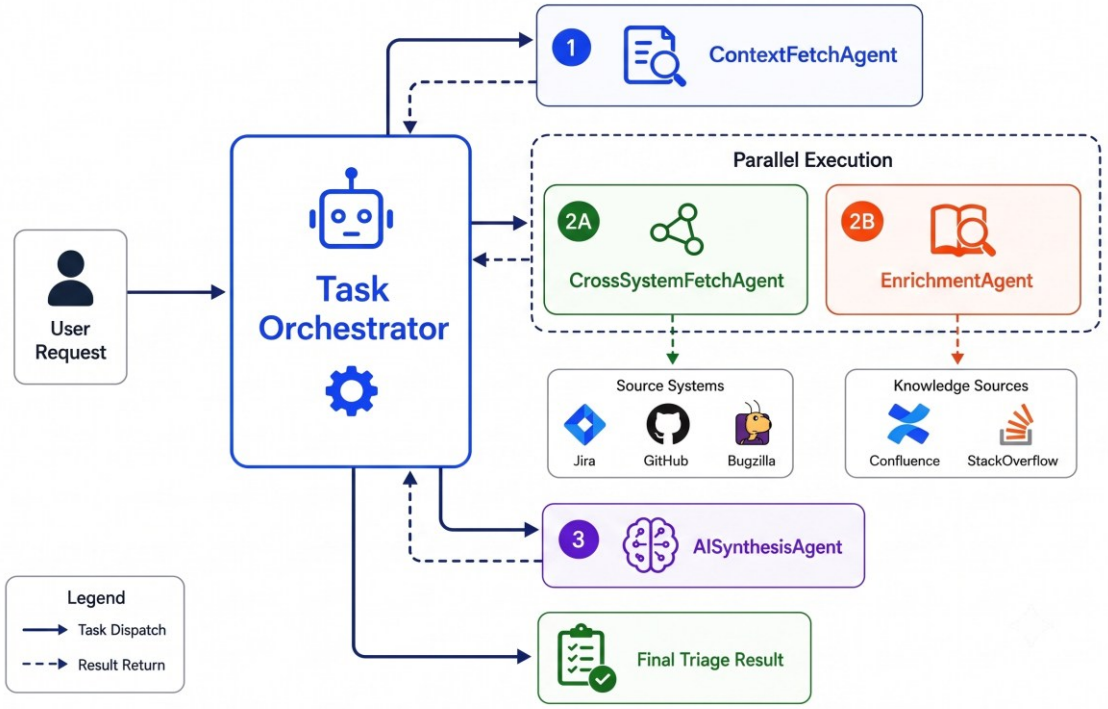
# Unified Bug List View



# Pipeline Coordination & Agent Roles

## TASK ORCHESTRATOR

- Starts the triage pipeline on every bug request
- Validates inputs and initializes shared pipeline context
- Sequences and triggers agents in the correct order
- Stores checkpoints after each agent stage
- Handles agent failures without breaking the pipeline
- Publishes final triage results to the UI




### 1 ContextFetchAgent

| | |
|---|---|
| Input | bug_id, source_id, case_id |
| Output | primary_ticket |
| Purpose | Fetches primary bug details |

### 2 CrossSystemFetchAgent

| | |
|---|---|
| Input | primary_ticket |
| Output | related_tickets |
| Purpose | Finds related bugs across systems |

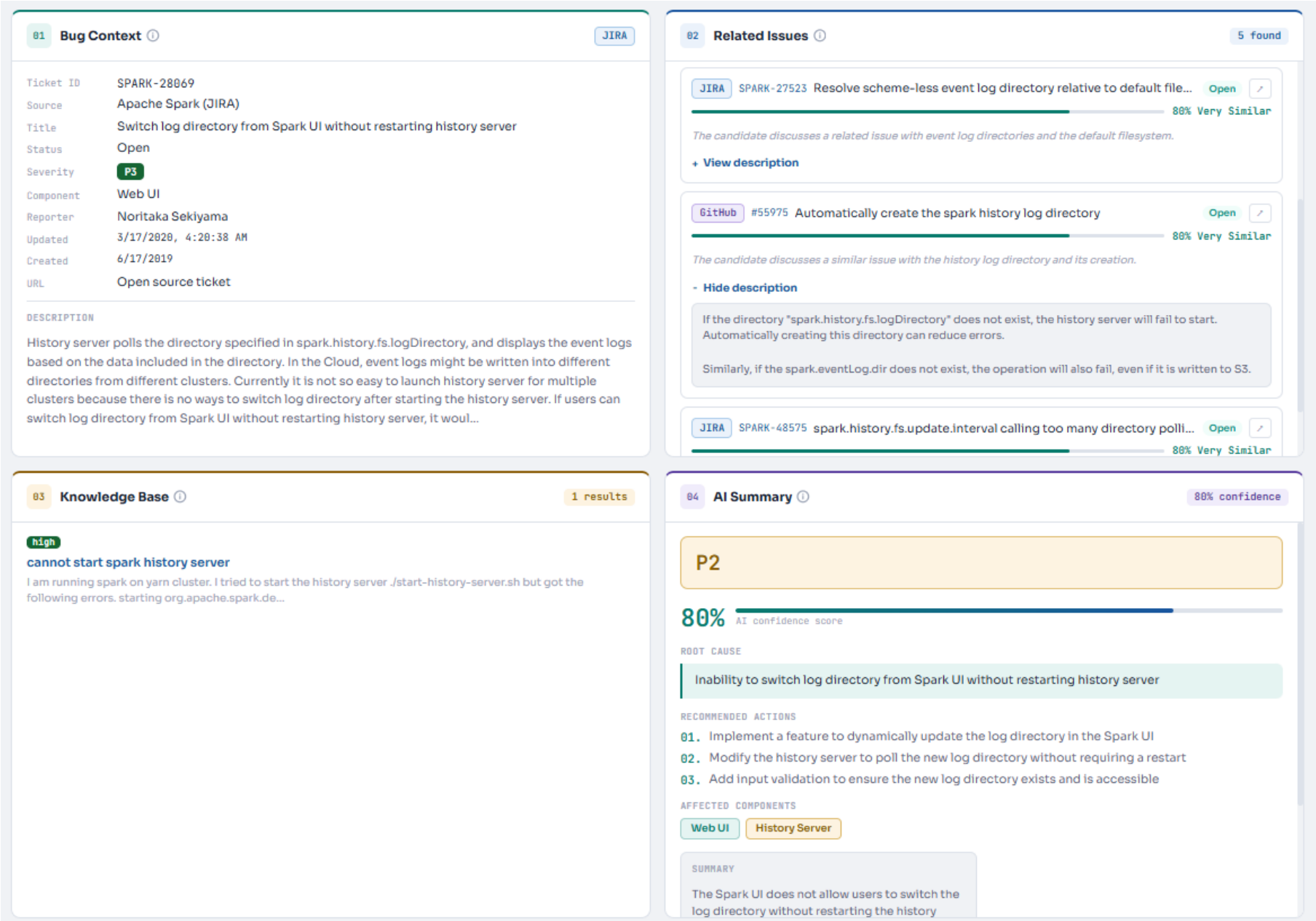
### 3 EnrichmentAgent

| | |
|---|---|
| Input | primary_ticket |
| Output | kb_articles |
| Purpose | Finds troubleshooting evidence |

### 4 AISynthesisAgent

| | |
|---|---|
| Input | primary_ticket + related_tickets + kb_articles |
| Output | ai_summary |
| Purpose | Combines all context into final triage result |

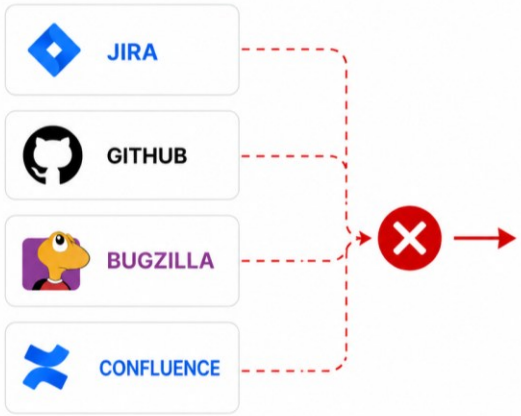
# Unified Triage View



# Connector Architecture & Adapter Pattern

## 1 THE PROBLEM

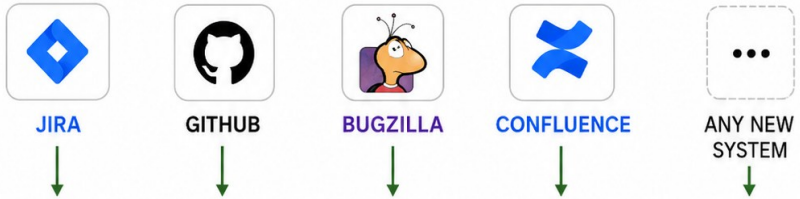
**Every system is different.**
Different APIs, formats and authentication.




Different formats, different APIs, hard to integrate and maintain.

## 2 OUR SOLUTION – ADAPTER PATTERN

Instead of making the pipeline understand every system, we make every system speak the **pipeline's language**.




### CONNECTOR REGISTRY (ADAPTER LAYER)

One common interface (BaseConnector) for all systems

| ID | Title | Description | Status | Type | Last Sync |
|---|---|---|---|---|---|
| JIR-01 | Jira Connector | Fetch issues & details | Active | REST API | 2m ago |

### COMMON TICKETDATA MODEL

One format. One schema. One language.

| ID | Title | Description | Status | Severity | Assignee | Source | Created At |
|---|---|---|---|---|---|---|---|
| TKT-1001 | Login issue | User unable to login | Open | High | John D. | Jira | 1h ago |

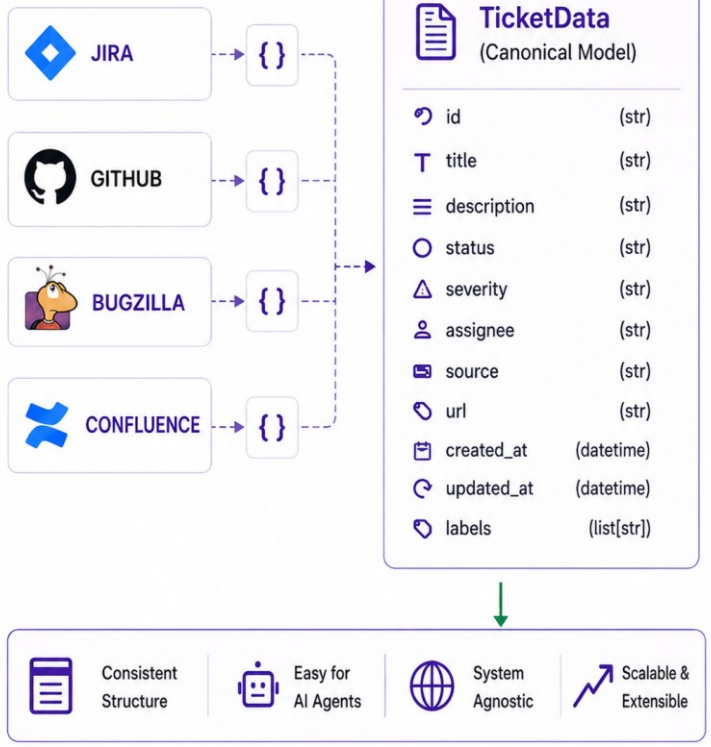
### AGENT PIPELINE (PLATFORM AGNOSTIC)

Agents don't know or care where the data came from.

- Context Fetch Agent
- Cross-System Fetch Agent
- Enrichment Agent
- AI Synthesis Agent

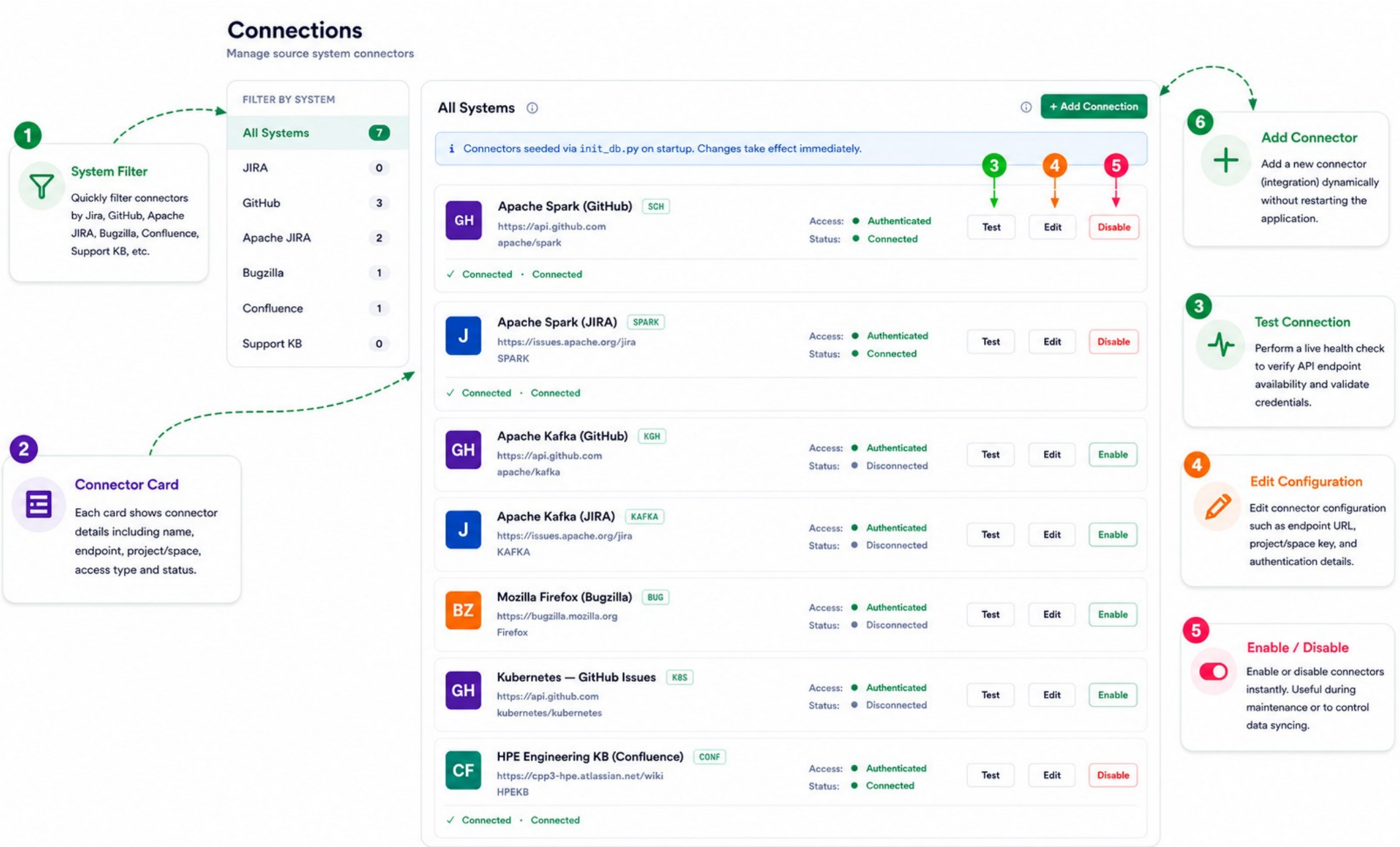
## 3 UNIFIED DATA (TICKETDATA MODEL)

Single TicketData Model for the Pipeline




Consistent Structure · Easy for AI Agents · System Agnostic · Scalable & Extensible

# Connector Configuration Dashboard

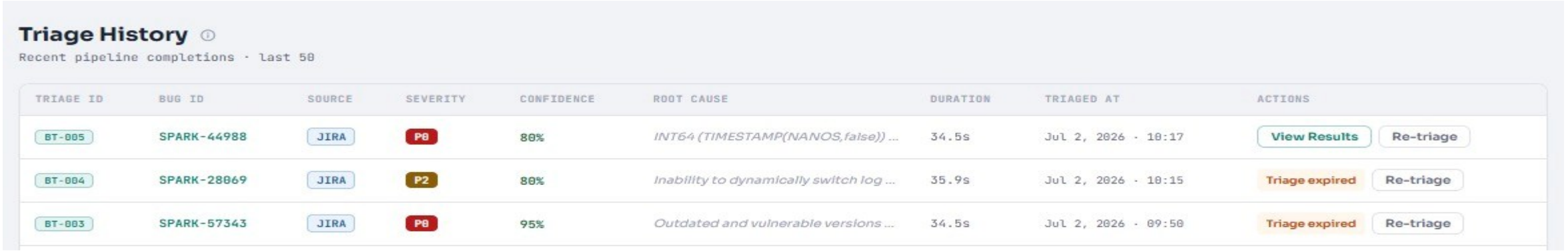


# Audit Log & Data Sovereignty

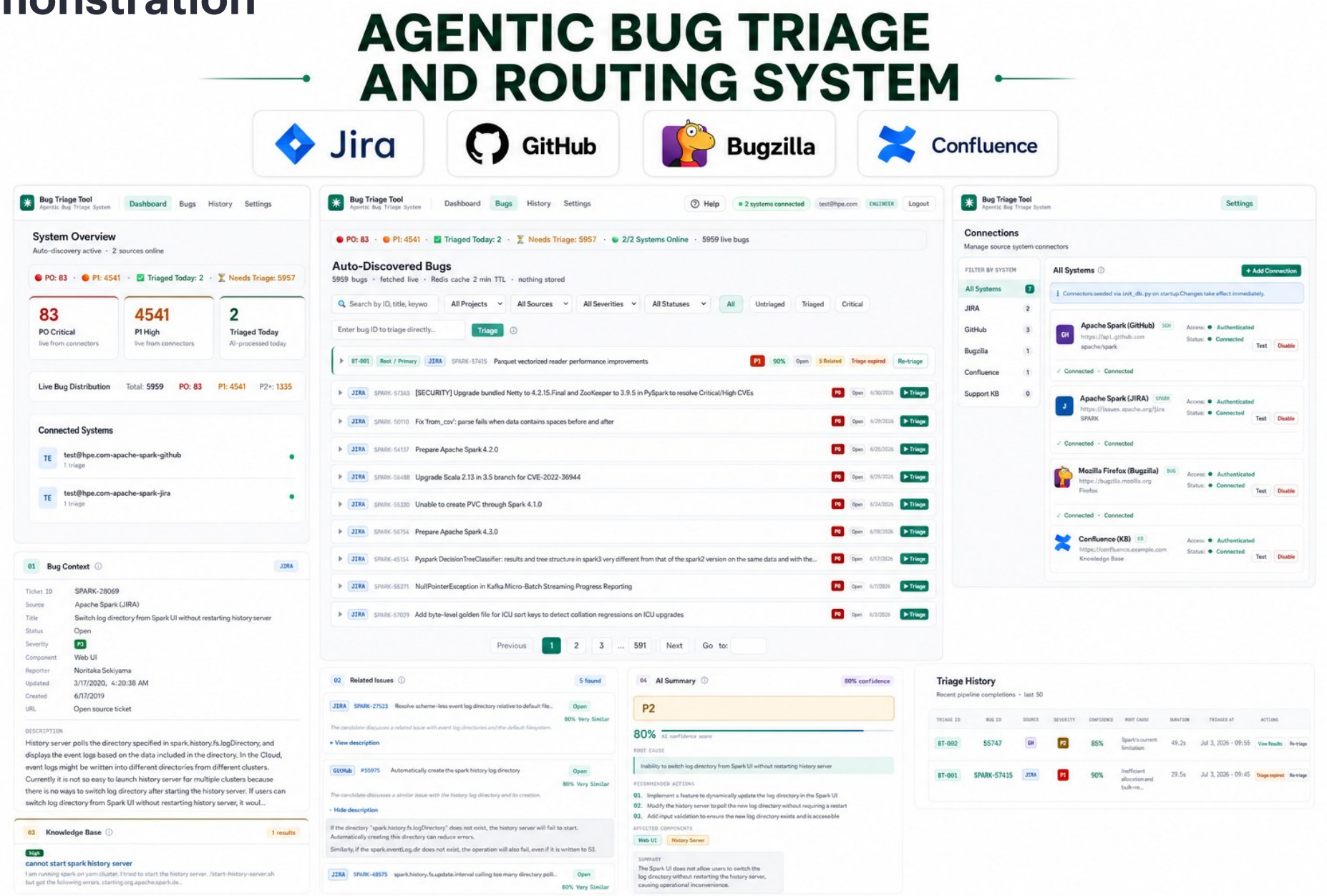
- **Zero-Residual Data Policy**
  - Raw bug content are held temporarily in pipeline context only
  - All raw data is purged immediately on run completion or failure

- **Metadata-Only Audit Trail**
  - Persistent logs capture only operational metadata: user ID, execution time, connectors queried
  - Raw bug texts and files are never written to the audit store.

- **Anonymized LLM Output**
  - Only high-level classifications are retained: unified severity (P0–P3), confidence score (0.0–1.0), truncated root cause hypothesis.
  - No raw LLM reasoning or source content is persisted.



# Live Demonstration




**Live Demo**: https://youtu.be/MUy3MSw08wI

# Key Learnings

**1. Multi-Agent Architecture**
Learned how to decompose complex workflows into specialized AI agents coordinated by a central orchestrator.

**2. Unified Bug Discovery**
Built a live discovery pipeline with parallel fetching and intelligent Redis caching for faster responses.

**3. LLM Integration**
Implemented robust LLM integration using validation, structured outputs, and fallback handling.

**4. Enterprise Data Sovereignty**
Designed a privacy-first architecture that preserves data sovereignty across enterprise systems.

**5. Adapter Design**
Applied the Adapter Pattern to integrate multiple enterprise platforms through a unified interface.

**6. Reliability & Fault Tolerance**
Enhanced system reliability using checkpointing, caching, and resilient execution strategies.

# Future Scope

**1. AI-Powered Semantic Correlation**
Improve duplicate detection and related bug discovery using semantic embeddings.

**2. Intelligent Role-Based Dashboards**
Deliver personalized dashboards and AI insights tailored to different organizational roles.

**3. Secure Credential Management**
Centralize connector authentication with enterprise-grade security and access control.

**4. Enterprise Connector Expansion**
Integrate additional enterprise platforms through the scalable Connector Registry.

**5. On-Premises AI Deployment**
Enable air-gapped deployments using locally hosted LLMs for regulated environments.

# Conclusion

### Unified Bug Intelligence Across Enterprise Systems

Connected JIRA, GitHub, Bugzilla, and Confluence into a single unified workspace.

### From Manual Investigation to Intelligent Automation

Automated context gathering, cross-system correlation, knowledge enrichment, and AI-assisted triage.

### One-Click End-to-End Bug Triage

Delivered unified bug context, related issues, supporting knowledge, and AI recommendations in under a minute.

### Enterprise-Ready Architecture

Scalable, extensible, read-only by design, with plug-and-play connector integrations.

# Thank You

# Tech Stack & Architecture Ecosystem

1. **Backend & API Layer -** Python 3.11, FastAPI, WebSockets
   **Purpose:** Powers asynchronous APIs and enables real-time communication between the frontend and backend.

2. **Async Execution Layer -** Apache Kafka, Asyncio
   **Purpose:** Supports event-driven workflows and parallel execution of AI agents.

3. **Data & Caching Layer -** PostgreSQL, Redis
   **Purpose:** Stores application metadata, connector configurations, and accelerates performance through caching.

4. **AI & Reasoning Layer -** Groq API, Llama 3 70B
   **Purpose:** Performs semantic understanding, bug correlation, severity prediction, and AI-assisted reasoning.

5. **Frontend Layer -** React, Vite, Tailwind CSS
   **Purpose:** Provides an interactive dashboard for live bug discovery, monitoring, and triage visualization.

6. **Enterprise Integrations -** Jira, GitHub, Bugzilla, Confluence
   **Purpose:** Retrieves live bug reports and knowledge-base information from enterprise platforms.

7. **Deployment Layer -** Docker, Docker Compose
   **Purpose:** Enables containerized, portable, and reproducible deployment across environments.

# Team Contribution

**Pulkit Jain:** API Gateway, Authentication & Authorization, REST APIs, Service Layer, WebSocket Communication

**Shivansh Gaur:** PostgreSQL Database Layer, Kafka Event Processing, Redis Caching, Docker & Infrastructure Setup

**Disha Jain:** Context Fetch Agent, AI Synthesis Agent, LLM Integration, Pipeline Orchestration

**Anuj Modani:** Cross-System Fetch Agent, Similarity Search, Knowledge Enrichment Agent, Related Issue Correlation

**Om Jain:** JIRA Integration, GitHub Integration, Bugzilla Integration, Confluence Integration, Connector Registry & Adapter Pattern